

# QR Code Motion Control

---

## 1. Introduction

This course mainly uses the robot's camera to obtain camera images, recognize QR code information, and control robot movement based on the QR code information.

## 2. Program Startup

### 2.1. Startup Commands

**For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,**

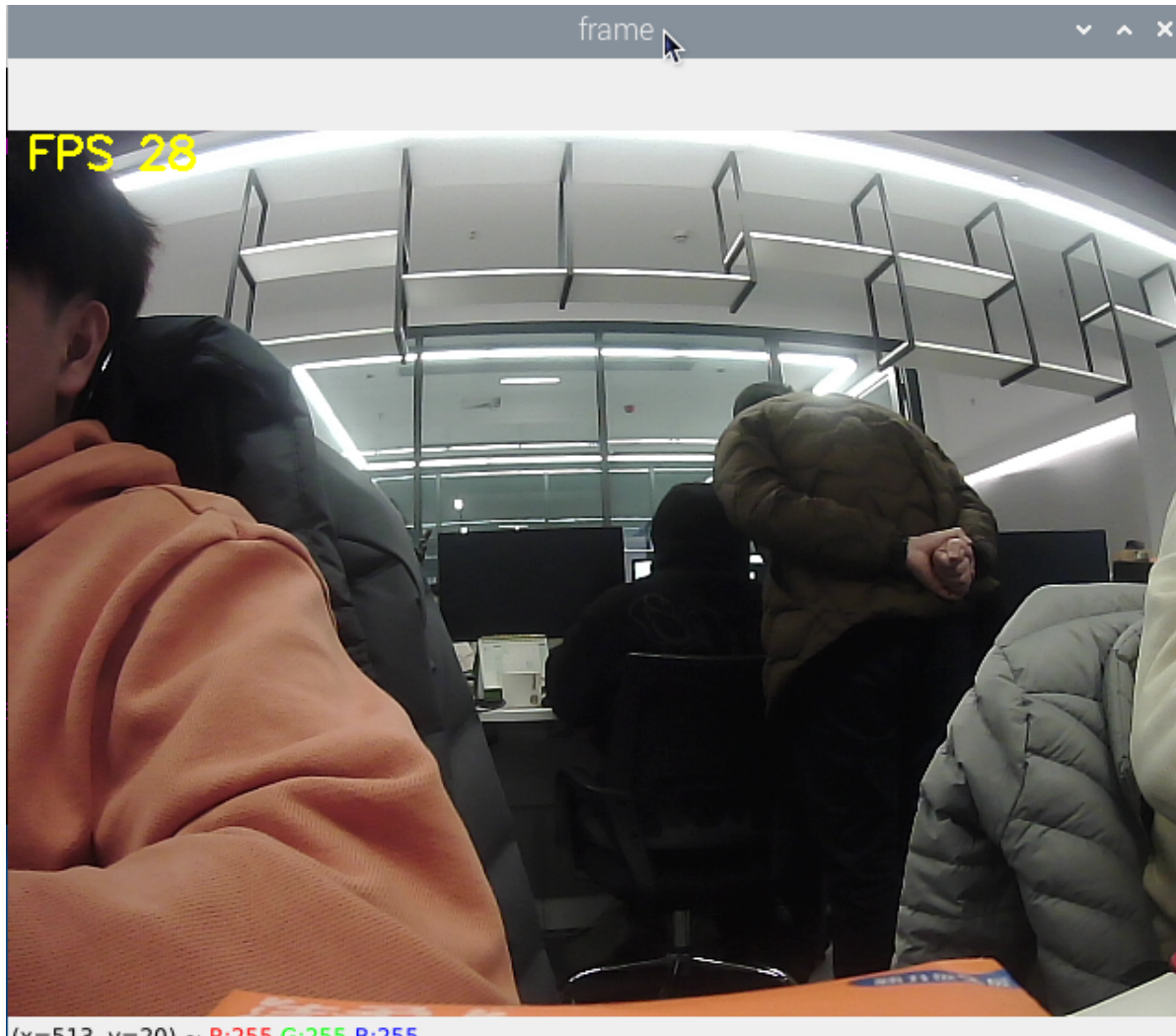
```
bringup_m3
```

**Open a terminal inside the container,**

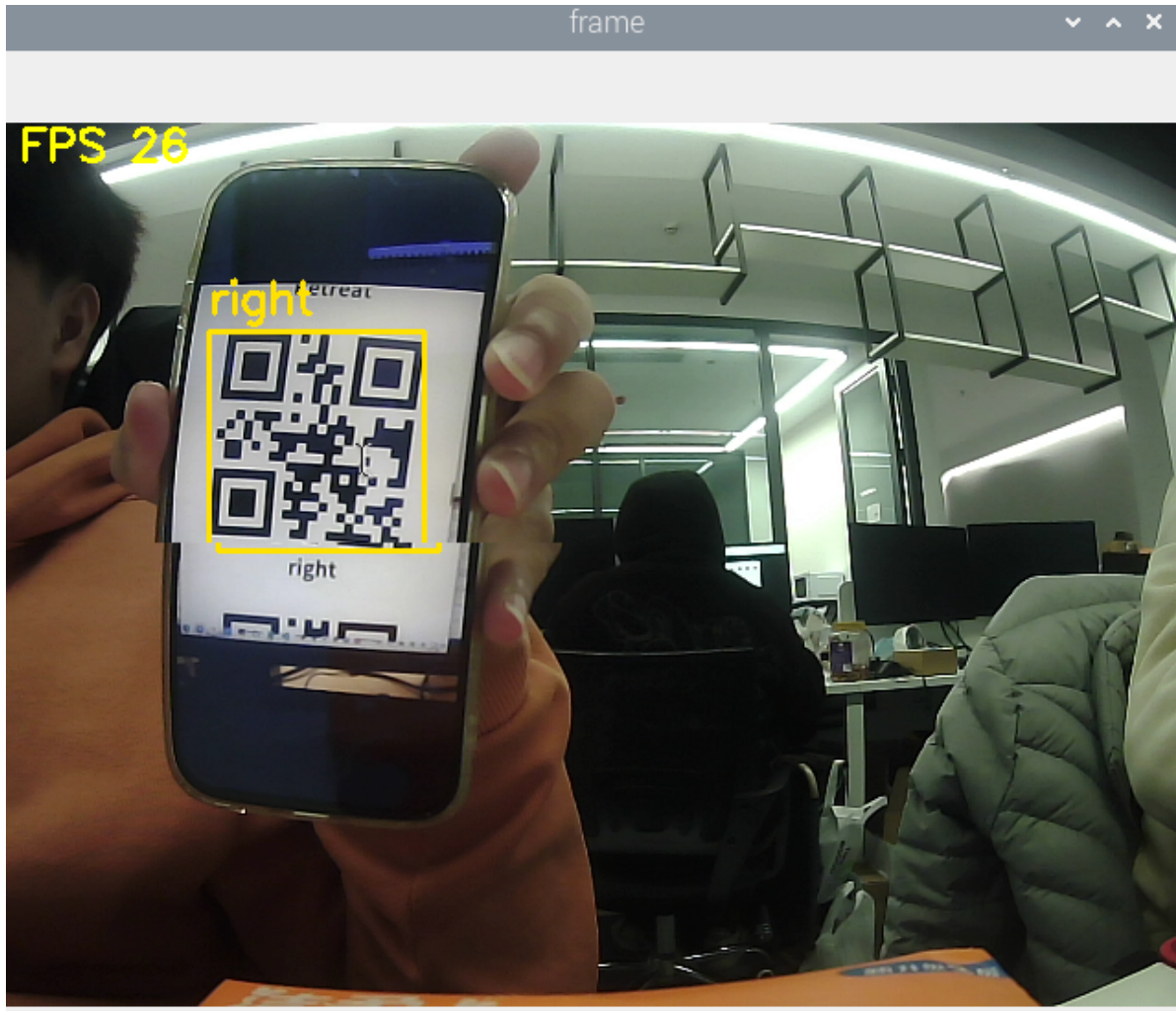
```
exec_m3
```

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=qrCtrl
```

Successfully display camera image



Enable QR code recognition and execute commands. Currently, the QR codes that can be recognized in the example are QRCode, with "forward" for moving forward, "back" for moving backward, "left" for turning left, "right" for turning right, and "stop" for stopping.



Press q to close the camera.

### 3. Core Code

Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/qrCtrl.py
```

Import QR code parsing library pyzbar

```
import pyzbar.pyzbar as pyzbar
from PIL import Image
```

Parse grayscale image to extract QR code information and image position from the image. If there is no QR code in the image, the information is None.

```

def detect_qrcode(image):
    # Convert to grayscale image
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    barcodes = pyzbar.decode(gray)
    for barcode in barcodes:
        # Extract QR code data and bounding box position
        (x, y, w, h) = barcode.rect
        barcodeData = barcode.data.decode("utf-8")
        barcodeType = barcode.type
        # print("[INFO] Found {} barcode: {}".format(barcodeType, barcodeData))
        car_control(barcodeData)
        return barcodeData, (x, y, w, h)
    return None, (0, 0, 0, 0)

```

Control robot movement based on info string commands.

```

def robot_action(self, data):
    if data == "forward":
        self.pub_vel(0.1, 0.0, 0.0)
        sleep(0.5)

    elif data == "back":
        self.pub_vel(-0.1, 0.0, 0.0)
        sleep(0.5)

    elif data == "left":
        self.pub_vel(0.1, 0.0, 1.0)
        sleep(0.5)

    elif data == "right":
        self.pub_vel(0.1, 0.0, -1.0)
        sleep(0.5)

    elif data == "stop":
        self.pub_vel(0.0, 0.0, 0.0)
        sleep(0.5)

```

Image processing program

```

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

    payload, (x, y, w, h) = self.detect_qrcode(frame)
    if payload != None:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 225, 255), 2)
        cv2.putText(frame, payload, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
(0, 225, 255), 2)
        self.robot_action(payload)
    else:
        self.pub_vel(0.0, 0.0, 0.0)

    cv2.imshow('frame', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        self.get_logger().info("Exiting program")
        cv2.destroyAllWindows()

```

```
rcipy.shutdown()  
exit(0)
```

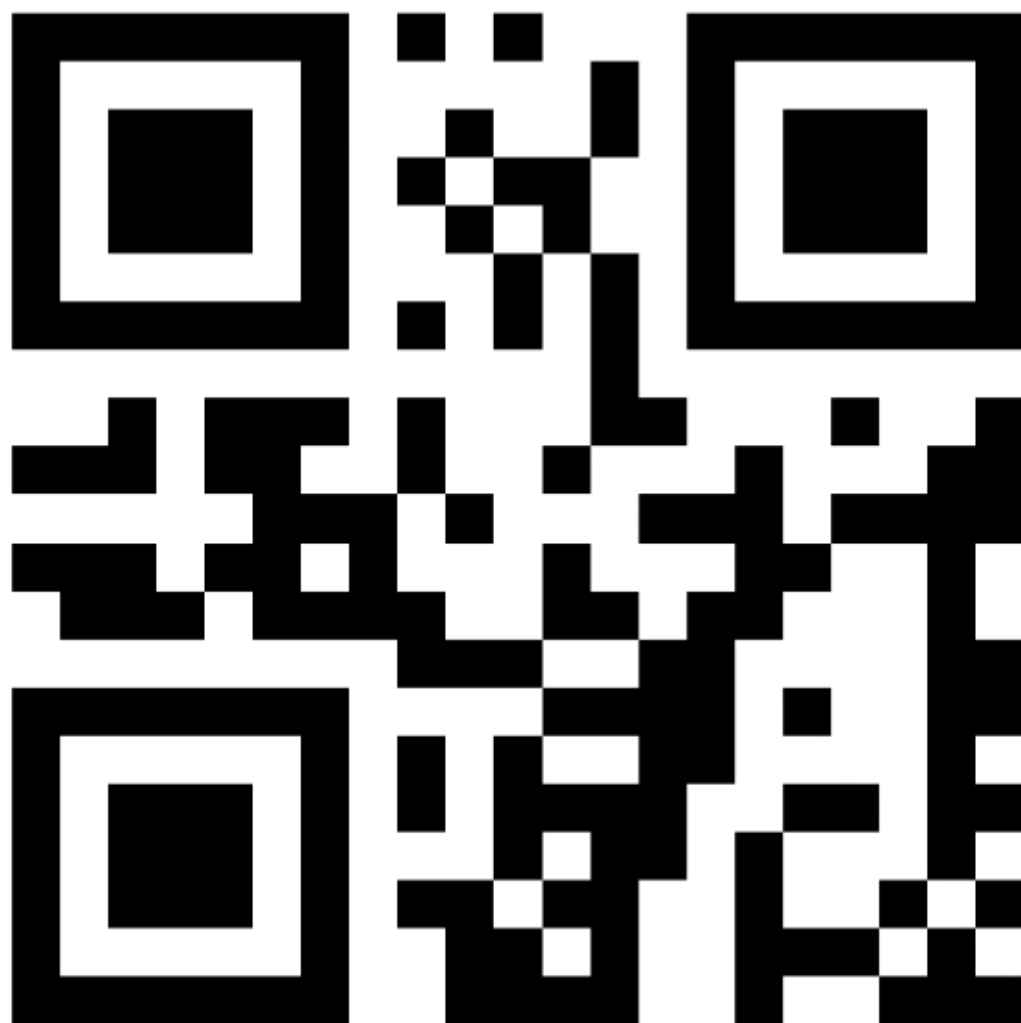
## Appendix (QR Code Images):



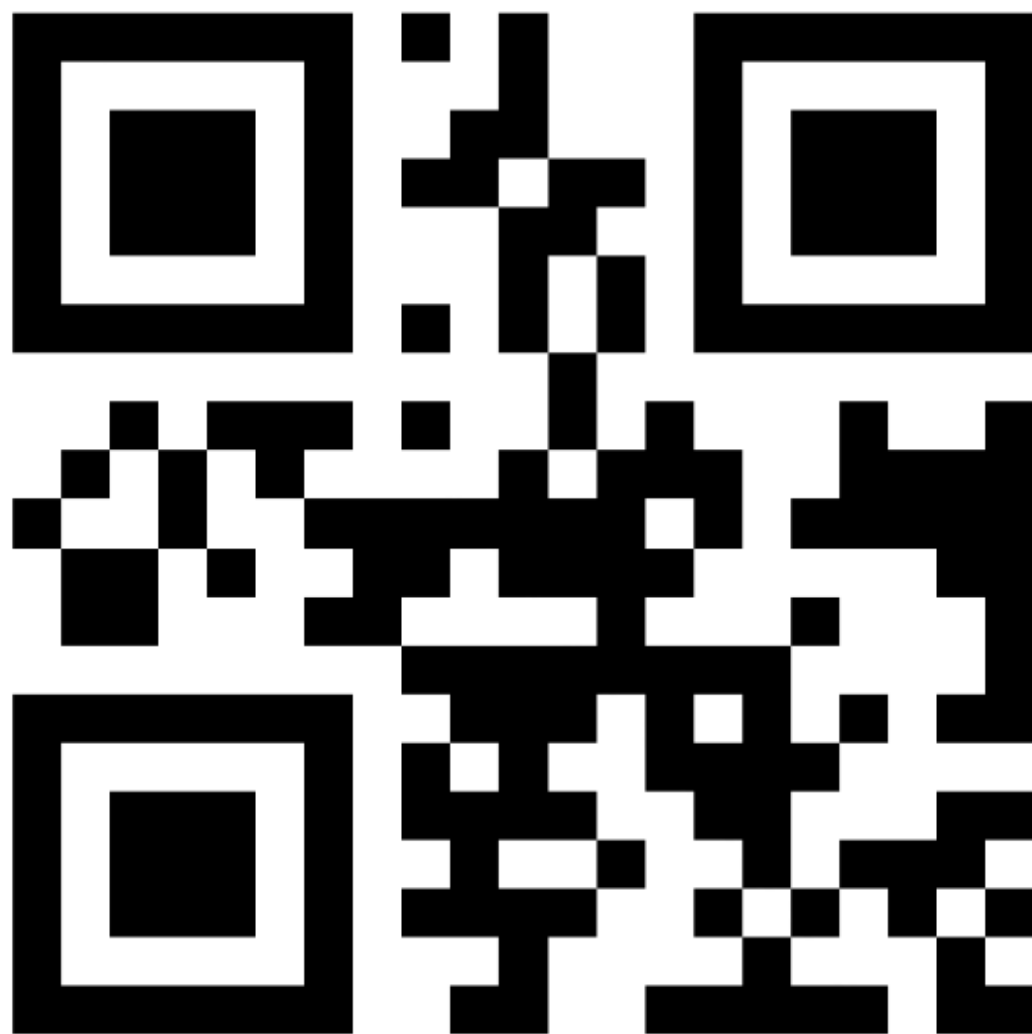
**forward**



**back**

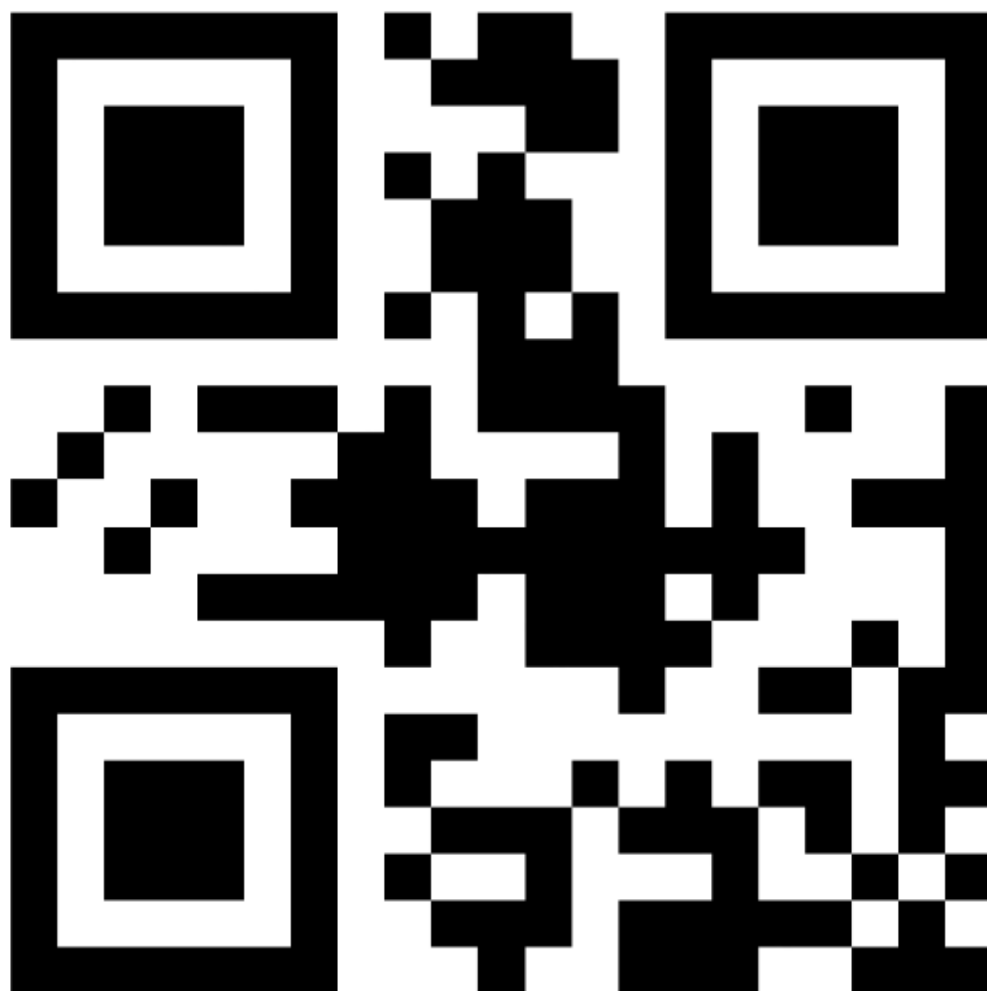


left



right





**stop**

